# Data Analysis for the Titanic Dataset

1. What was the overall **survival rate**?
2. How did **survival** differ by **gender**?
3. How did **survival** differ by **passenger class**?
4. How did survival break down by **both** class and **gender**?
5. What was the **survival rate** by **age group**?

In [133…
```
import pandas as pd
```

In [134…
```
df = pd.read_csv('Titanic-Dataset.csv')
```

In [135…
```
df
```

Out[135…

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | NaN | S |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th… | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C85 | C |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | NaN | S |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C123 | S |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | NaN | S |
| **…** | … | … | … | … | … | … | … | … | … | … | … | … |
| **886** | 887 | 0 | 2 | Montvila, Rev. Juozas | male | 27.0 | 0 | 0 | 211536 | 13.0000 | NaN | S |
| **887** | 888 | 1 | 1 | Graham, Miss. Margaret Edith | female | 19.0 | 0 | 0 | 112053 | 30.0000 | B42 | S |
| **888** | 889 | 0 | 3 | Johnston, Miss. Catherine Helen "Carrie" | female | NaN | 1 | 2 | W./C. 6607 | 23.4500 | NaN | S |
| **889** | 890 | 1 | 1 | Behr, Mr. Karl Howell | male | 26.0 | 0 | 0 | 111369 | 30.0000 | C148 | C |
| **890** | 891 | 0 | 3 | Dooley, Mr. Patrick | male | 32.0 | 0 | 0 | 370376 | 7.7500 | NaN | Q |

891 rows × 12 columns

In [136…
```
df.describe()
```

Out[136…

| | PassengerId | Survived | Pclass | Age | SibSp | Parch | Fare |
|---|---|---|---|---|---|---|---|
| **count** | 891.000000 | 891.000000 | 891.000000 | 714.000000 | 891.000000 | 891.000000 | 891.000000 |
| **mean** | 446.000000 | 0.383838 | 2.308642 | 29.699118 | 0.523008 | 0.381594 | 32.204208 |
| **std** | 257.353842 | 0.486592 | 0.836071 | 14.526497 | 1.102743 | 0.806057 | 49.693429 |
| **min** | 1.000000 | 0.000000 | 1.000000 | 0.420000 | 0.000000 | 0.000000 | 0.000000 |
| **25%** | 223.500000 | 0.000000 | 2.000000 | 20.125000 | 0.000000 | 0.000000 | 7.910400 |
| **50%** | 446.000000 | 0.000000 | 3.000000 | 28.000000 | 0.000000 | 0.000000 | 14.454200 |
| **75%** | 668.500000 | 1.000000 | 3.000000 | 38.000000 | 1.000000 | 0.000000 | 31.000000 |
| **max** | 891.000000 | 1.000000 | 3.000000 | 80.000000 | 8.000000 | 6.000000 | 512.329200 |

In [137…
```
df.columns
```

Out[137…
```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

In [138…
```
df.info()
```

Talha Shahid

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

## Handling Missing Data

Before we can analyze the data, we must "clean" it. When we ran `df.info()`, we saw that the `Age` and `Embarked` columns have missing values (`NaN`). Many analysis functions will fail or produce incorrect results if they encounter `NaN` data.

We need to "impute" (fill in) these missing values using a reasonable strategy.

### For the `Age` Column (Numerical Data): Fill with Median

- **Problem:** The `Age` column is numerical but has many missing values. We can't just drop these rows, as we would lose a significant portion of our data.
- **Solution:** We will fill the `NaN` values with the **median** age.
- **Why Median (and not Mean)?**
  - The **mean** (average) is highly sensitive to outliers. A few very old passengers could skew the mean age upwards, making it a poor representation of the "typical" passenger.
  - The **median** is the middle value (the 50th percentile) of all the ages. It is "robust" to outliers and gives a much better estimate of the central tendency for this dataset. We will calculate the median age from all the *known* ages and use that single number to fill in all the blanks.

### For the `Embarked` Column (Categorical Data): Fill with Mode

- **Problem:** The `Embarked` column is missing just two values.
- **Solution:** We will fill these with the **mode** (the most frequent value).
- **Why Mode?**
  - `Embarked` is categorical data (S, C, or Q), not numerical. We cannot calculate a mean or median.
  - The mode is the most common port of embarkation. Since only two values are missing, the safest and most statistically sound assumption is that these passengers embarked from the same port as the vast majority of others. This has a minimal impact on the dataset's integrity.

In [139…
```
df['Age'] = df['Age'].fillna(df['Age'].median())
```

In [140…
```
df
```

Out[140…

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | NaN | S |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th… | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C85 | C |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | NaN | S |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C123 | S |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | NaN | S |
| **…** | … | … | … | … | … | … | … | … | … | … | … | … |
| **886** | 887 | 0 | 2 | Montvila, Rev. Juozas | male | 27.0 | 0 | 0 | 211536 | 13.0000 | NaN | S |
| **887** | 888 | 1 | 1 | Graham, Miss. Margaret Edith | female | 19.0 | 0 | 0 | 112053 | 30.0000 | B42 | S |
| **888** | 889 | 0 | 3 | Johnston, Miss. Catherine Helen "Carrie" | female | 28.0 | 1 | 2 | W./C. 6607 | 23.4500 | NaN | S |
| **889** | 890 | 1 | 1 | Behr, Mr. Karl Howell | male | 26.0 | 0 | 0 | 111369 | 30.0000 | C148 | C |
| **890** | 891 | 0 | 3 | Dooley, Mr. Patrick | male | 32.0 | 0 | 0 | 370376 | 7.7500 | NaN | Q |

891 rows × 12 columns

In [141…
```
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
```

In [142…
```
df
```

Out[142…

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | NaN | S |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C85 | C |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | NaN | S |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C123 | S |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | NaN | S |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **886** | 887 | 0 | 2 | Montvila, Rev. Juozas | male | 27.0 | 0 | 0 | 211536 | 13.0000 | NaN | S |
| **887** | 888 | 1 | 1 | Graham, Miss. Margaret Edith | female | 19.0 | 0 | 0 | 112053 | 30.0000 | B42 | S |
| **888** | 889 | 0 | 3 | Johnston, Miss. Catherine Helen "Carrie" | female | 28.0 | 1 | 2 | W./C. 6607 | 23.4500 | NaN | S |
| **889** | 890 | 1 | 1 | Behr, Mr. Karl Howell | male | 26.0 | 0 | 0 | 111369 | 30.0000 | C148 | C |
| **890** | 891 | 0 | 3 | Dooley, Mr. Patrick | male | 32.0 | 0 | 0 | 370376 | 7.7500 | NaN | Q |

891 rows × 12 columns

In [143…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          891 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     891 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Talha Shahid

## 1. What was the overall survival rate?

In [144…
```
overAllSurvivalRate = df['Survived'].mean() * 100
```

In [145…
```
overAllSurvivalRate
```

Out[145…
```
np.float64(38.38383838383838)
```

## Why the Mean of 'Survived' Equals the Survival Rate

Calculating the `mean()` of the Titanic's **`Survived`** column works as a shortcut for the survival rate because of the way the data is encoded.

This method relies on a clever mathematical property of **binary** columns (columns containing only 0s and 1s).

1. **Binary Encoding:** In the Titanic dataset, the `Survived` column is binary:
    - **0** = Did Not Survive
    - **1** = Survived
2. **The Math of the Mean:** The formula for the mean (or average) is:

$$\text{Mean} = \frac{\text{Sum of all values}}{\text{Number of all values}}$$

3. **Applying to 'Survived':** Let's see what happens when we apply this formula to the `Survived` column:
    - **`Number of all values`**: This is the total number of passengers in the dataset.
    - **`Sum of all values`**: This is the sum of all the 0s and 1s in the column. Since adding 0 doesn't change the total, the sum is simply the **total count of 1s**.
4. **The "Aha!" Moment:**
    - The **`Sum of all values`** is just the **total number of survivors**.
    - The **`Number of all values`** is the **total number of passengers**.

Therefore, the calculation becomes:

$$\text{Mean} = \frac{\text{Total Number of Survivors}}{\text{Total Number of Passengers}}$$

This is the exact definition of the **survival rate**.

### Simple Example

Imagine 5 passengers with the following `Survived` data: `[1, 0, 0, 1, 0]`

- **Survival Rate:** There are 2 survivors out of 5 passengers. The rate is $\frac{2}{5} = 0.4$ (or $40\%$).
- **Mean Calculation:**
  - $\text{Sum} = 1 + 0 + 0 + 1 + 0 = 2$
  - $\text{Count} = 5$
  - $\text{Mean} = \frac{2}{5} = 0.4$

Both methods give the exact same result.

## Manually Calculating the `Survival Rate`

### Caluclating `Total Number of Survivors`

In [146…
```
survivalOnly = df[df['Survived']==1]
```

In [147…
```
survivalOnlyCount =survivalOnly['Survived'].value_counts()
```

In [148…
```
survivalOnlyCount
```

Out[148…
```
Survived
1    342
Name: count, dtype: int64
```

### Caluclating `Total Passengers`

In [149…
```
TotalPassengers = len(df)
```

In [150…
```
TotalPassengers
```

Out[150… 891

Talha Shahid

$$\text{Suvival Rate} = \frac{\text{SurvivalOnlyCount}}{\text{TotalPassengers}}$$

In [151…
```
survivalRate = survivalOnlyCount/TotalPassengers
```

In [152…
```
survivalRate
```

Out[152…
```
Survived
1    0.383838
Name: count, dtype: float64
```

# 2. How did survival differ by gender?

This tests the **"women and children first"** idea and is one of the most well-known insights.

In [153…
```
genderSurvival = df.groupby('Sex')['Survived'].mean()
```

In [154…
```
genderSurvival
```

Out[154…
```
Sex
female    0.742038
male      0.188908
Name: Survived, dtype: float64
```

### Why Grouped Survival Rates Don't Sum to 100%

When you calculate the survival rate for different groups separately (like for males and females), the resulting percentages do not sum to 1.0 (or 100%).

This is because they are **not** parts of the same whole. They are two **separate, independent statistics**.

---

#### What the Rates Represent

Let's imagine the calculation gives these results:

- **Female Survival Rate: 0.74 (74%)**
- **Male Survival Rate: 0.19 (19%)**

Here is what each number actually means:

1. **The 74% (Female Rate):** This percentage answers the question, "Out of all the females, what portion survived?"
    - It's calculated as:

$$\frac{\text{Number of Females who Survived}}{\text{Total Number of Females}}$$

    - The "whole" for this statistic is the **total number of females**. The other 26% of *this* whole $(1.0 - 0.74)$ represents the females who did *not* survive.
2. **The 19% (Male Rate):** This percentage answers the question, "Out of all the males, what portion survived?"
    - It's calculated as:

$$\frac{\text{Number of Males who Survived}}{\text{Total Number of Males}}$$

    - The "whole" for this statistic is the **total number of males**. The other 81% of *this* whole $(1.0 - 0.19)$ represents the males who did *not* survive.

Since each percentage is calculated using a different denominator (a different "whole"), there is no mathematical reason they should add up to 100%.

---

### What *Would* Sum to 100%

Percentages only sum to 100% when they represent different slices of the *same* pie.

For example, if you asked, "What percentage of *all passengers* were female?" and "What percentage of *all passengers* were male?" Those two numbers would sum to 100%, because they are both parts of the same single whole: the **total number of passengers**.

## 3. How did survival differ by passenger class?

This question explores the impact of socio-economic status on survival. (`1 = 1st Class`, `2 = 2nd`, `3 = 3rd`).

In [155…
```
classSurvival = df.groupby('Pclass')['Survived'].mean()
```

In [156…
```
classSurvival
```

Out[156…
```
Pclass
1    0.629630
2    0.472826
3    0.242363
Name: Survived, dtype: float64
```

## 4. How did survival break down by both class and gender?

This is a more advanced question that looks at the interaction between two key variables. This shows how strongly `"class"` and `"gender"` combined to predict survival.

In [157…
```
classGenderSurvival = df.groupby(['Pclass', 'Sex'])['Survived'].mean()
```

In [158…
```
classGenderSurvival
```

Out[158…
```
Pclass  Sex
1       female    0.968085
        male      0.368852
2       female    0.921053
        male      0.157407
3       female    0.500000
        male      0.135447
Name: Survived, dtype: float64
```

## 5. What was the survival rate by age group?

Since `Age` is a continuous number, it's much more insightful to group people into bins (like `'Child'`, `'Teen'`, `'Adult'`). We can do this with `pd.cut`.

In [159…
```
# Bins: 0-16 (Child), 17-25 (Young Adult), 26-50 (Adult), 51+ (Senior)
ageBins = [0, 16, 25, 50, 100]
ageLabels = ['Child', 'Young Adult', 'Adult', 'Senior']

# 2. Create the new 'AgeGroup' column
df['AgeGroup'] = pd.cut(df['Age'], bins=ageBins, labels=ageLabels, right=True)
```

In [160…
```
df
```

Out[160…

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked | AgeGroup |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | NaN | S | Young Adult |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C85 | C | Adult |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | NaN | S | Adult |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C123 | S | Adult |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | NaN | S | Adult |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **886** | 887 | 0 | 2 | Montvila, Rev. Juozas | male | 27.0 | 0 | 0 | 211536 | 13.0000 | NaN | S | Adult |
| **887** | 888 | 1 | 1 | Graham, Miss. Margaret Edith | female | 19.0 | 0 | 0 | 112053 | 30.0000 | B42 | S | Young Adult |
| **888** | 889 | 0 | 3 | Johnston, Miss. Catherine Helen "Carrie" | female | 28.0 | 1 | 2 | W./C. 6607 | 23.4500 | NaN | S | Adult |
| **889** | 890 | 1 | 1 | Behr, Mr. Karl Howell | male | 26.0 | 0 | 0 | 111369 | 30.0000 | C148 | C | Adult |
| **890** | 891 | 0 | 3 | Dooley, Mr. Patrick | male | 32.0 | 0 | 0 | 370376 | 7.7500 | NaN | Q | Adult |

891 rows × 13 columns

In [161…
```
AgeGroupSurvial = df.groupby('AgeGroup', observed=True)['Survived'].mean()
```

In [162…
```
AgeGroupSurvial
```

Out[162…
```
AgeGroup
Child          0.550000
Young Adult    0.343284
Adult          0.372624
Senior         0.343750
Name: Survived, dtype: float64
```

**binning** or **discretization** is a common technique, where you convert a continuous numerical variable (like `'Age'`) into a discrete categorical variable (like `'Child'` or `'Adult'`).

## Step 1: Define the Bins and Labels

```
# Bins: 0-16 (Child), 17-25 (Young Adult), 26-50 (Adult), 51+ (Senior)
ageBins = [0, 16, 25, 50, 100]
ageLabels = ['Child', 'Young Adult', 'Adult', 'Senior']
```

- `ageBins`: This list defines the edges of your age categories. You need N+1 edges to create N bins. Here, the 5 numbers create 4 distinct bins.
- `ageLabels`: This list provides the string names for each bin. The number of labels must be one less than the number of bin edges.

This setup will create the following groups:

- **Bin 1:** Ages between 0 and 16
- **Bin 2:** Ages between 16 and 25
- **Bin 3:** Ages between 25 and 50
- **Bin 4:** Ages between 50 and 100 (100 is chosen as an upper limit to include all passengers)

## Step 2: Create the new `'AgeGroup'` Column

```
df['AgeGroup'] = pd.cut(df['Age'], bins=ageBins, labels=ageLabels, right=True)
```

This line is the core of the operation. It uses the `pd.cut()` function to "cut" the ages into the bins and then assigns the results to a new column called AgeGroup.

- `df['Age']`: This is the input data—the continuous `'Age'` column that you want to categorize.
- `bins=ageBins`: This tells the function to use the bin edges you defined in `ageBins`.
- `labels=ageLabels`: This tells the function to use the string names from `ageLabels` for each bin.
- `right=True`: This is an important parameter. It means that the right side of the bin is inclusive.
  - `Child: (0, 16]` → Ages greater than 0, up to and including 16.
  - `Young Adult: (16, 25]` → Ages greater than 16, up to and including 25.
  - `Adult: (25, 50]` → Ages greater than 25, up to and including 50.
  - `Senior: (50, 100]` → Ages greater than 50, up to and including 100.

After this line runs, the df has a new column, `AgeGroup`, where every passenger is labeled as `'Child'`, `'Young Adult'`, `'Adult'`, or `'Senior'`.

## Step 3: Calculate Survival Rate by Group

```
AgeGroupSurvial = df.groupby('AgeGroup', observed=True)['Survived'].mean()
```

This line performs the analysis.

- `df.groupby('AgeGroup', ...)`: This first groups all the passengers based on the new `'AgeGroup'` categories. You now have 4 separate groups of passengers.
- `observed=True`: This is a performance and-cleanup setting. It tells groupby to only create groups for categories that are actually present in the `'AgeGroup'` column. (For example, if your dataset had no one in the `'Senior'` bin, this would prevent an empty `'Senior'` group from appearing in the output).
- `['Survived']`: From those 4 groups, it selects just the `'Survived'` column.
- `.mean()`: Finally, it calculates the mean (average) of the `'Survived'` column for each group. As you know, because `'Survived'` is binary (0 or 1), its mean is the same as its survival rate.

The final variable, `AgeGroupSurvial`, is a pandas Series that holds the survival rate for each of the four age groups, allowing you to easily compare them.

Talha Shahid